

MINI PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM

Console-Based Record-Management Application

Assignment 1 – Mini Project

MCA Semester I – Python Programming & Relational Database

Submitted by:

Pranjal Wadhawane

Technology: Python 3

Storage: JSON File

Development Environment: Visual Studio Code

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Technologies and Concepts Used
6. System Requirements
7. System Design and Working
8. Functional Modules
9. File I/O and Data Persistence
10. Exception Handling
11. Project Structure
12. Testing and Sample Output
13. Advantages
14. Limitations
15. Future Enhancements
16. Conclusion
17. References

1. Introduction

The Student Record Management System is a console-based Python application developed to manage student information in a simple and organized manner. The application provides a menu-driven interface through which a user can add, view, search, update, and delete student records. Student data is stored in a JSON file so that records remain available after the program is closed and reopened.

The project is designed as a practical implementation of fundamental Python programming concepts required for the MCA Semester I assignment. It combines data types, lists and dictionaries, conditional statements, loops, functions, exception handling, and file I/O within one working application.

2. Problem Statement

Managing student information manually can become difficult when records increase in number. A basic record-management application is therefore required to provide a simple way to store and maintain student details while reducing repetitive manual work.

The proposed system solves this problem by providing a console-based menu that allows users to perform common record-management operations and save the information persistently using a JSON file.

3. Objectives

- Develop a fully working menu-driven console application in Python.
- Allow users to add new student records.
- Display all stored student records.
- Search for a student using Student ID.
- Update an existing student record.
- Delete an existing student record.
- Use functions to divide the program into manageable modules.

- Handle invalid input and file-related errors using exception handling.
- Store and retrieve records using JSON-based File I/O.
- Maintain a clean and understandable project structure suitable for GitHub submission.

4. Scope of the Project

The project is intended for small-scale student record management through a local command-line interface. It is suitable for demonstrating fundamental programming and data-handling concepts. The current version does not require a database server, web application, or external package.

5. Technologies and Concepts Used

Technology / Concept	Purpose
Python 3	Main programming language used to develop the application.
JSON	File format used to store student records persistently.
Visual Studio Code	Code development and testing environment.
Variables and Data Types	Store student IDs, names, ages, courses, and email information.
Lists and Dictionaries	Represent and organize multiple student records.
Conditional Statements	Control menu selection and validation logic.
Loops	Repeatedly display the menu and process user operations.
Functions	Separate major operations into reusable program units.
Exception Handling	Handle invalid input and file/JSON errors safely.
File I/O	Read from and write records to the JSON file.

6. System Requirements

Hardware Requirements

- Computer or laptop with sufficient storage for Python and project files.
- Keyboard and display for console interaction.

Software Requirements

- Python 3.x
- Visual Studio Code or another Python-compatible editor
- macOS, Windows, or Linux operating system

7. System Design and Working

The application follows a simple menu-driven design. When the program starts, it loads existing records from the JSON file. The user is then shown a menu and selects an operation. The selected operation is handled by a dedicated function. After an operation that changes data, the updated records are written back to the JSON file.

1. Program starts.
2. Existing records are loaded from records.json.
3. The main menu is displayed.
4. The user selects an operation from the available choices.
5. The corresponding function performs the requested operation.
6. Errors such as invalid input or invalid JSON are handled using exception handling.

7. The menu is displayed again until the user selects Exit.

8. Functional Modules

Module	Description
Add Student Record	Accepts student information and adds a new record to the collection.
View All Records	Displays the student records currently stored in the system.
Search Student	Searches for a record using the Student ID.
Update Student Record	Modifies information belonging to an existing Student ID.
Delete Student Record	Removes a selected student record from the stored data.
Exit	Closes the application after completing the required operations.

9. File I/O and Data Persistence

The project uses the file records.json as its persistent storage file. JSON is suitable for this mini project because it is lightweight, human-readable, and directly supported by Python through the built-in json module.

The application reads the JSON file when records need to be loaded and writes updated records after changes. This ensures that student information is not lost when the program is closed.

10. Exception Handling

Exception handling is included to prevent the program from terminating unexpectedly when a user provides invalid input or when the records file cannot be read correctly. Typical cases include invalid JSON content, file access/read errors, and invalid values entered through the console.

- FileNotFoundError – handled when the records file does not exist.
- JSONDecodeError – handled when the JSON file contains invalid JSON data.
- OSError – handled for general file reading problems.
- Input validation – used to reduce errors caused by invalid user input.

11. Project Structure

```
Student-Record-Management-
|
├── screenshots/
│   ├── 01_main_menu.png
│   ├── 02_add_student.png
│   ├── 03_view_students.png
│   ├── 04_search_student.png
│   ├── 05_update_student.png
│   ├── 06_delete_student.png
│   └── 07_exit.png
├── records.json
├── student_record_management.py
├── README.md
└── REPORT.md
```

The screenshots folder is used to document the working application. The Python source file contains the program logic, records.json stores the data, README.md provides project information and usage instructions, and REPORT.md is the assignment documentation.

12. Testing and Sample Output

The application was tested through the console by executing the main menu and performing the major record-management operations. The screenshots included with the project provide visual evidence of the application running in Visual Studio Code.

- Program startup and menu display.
- Adding a student record.
- Viewing stored records.
- Searching by Student ID.
- Updating an existing student record.
- Deleting a student record.
- Exiting the application.

Sample Console Menu

```
===== STUDENT RECORD MANAGEMENT SYSTEM =====
1. Add Student Record
2. View All Records
3. Search Student Record
4. Update Student Record
5. Delete Student Record
6. Exit
```

Enter your choice (1-6):

Sample Add Operation

```
Enter student name: Pranjali Wadhawane
Enter student age: 21
Enter course: MCA
Enter email: student@example.com
Student record added successfully.
```

13. Advantages

- Simple and easy-to-use console interface.
- Provides all basic CRUD-style record operations required for the assignment.
- Uses functions for better organization and maintainability.
- Stores records persistently using JSON File I/O.
- Uses built-in Python functionality, keeping the project lightweight.
- Includes exception handling for common runtime and file-related problems.
- Easy to run on Windows, macOS, and Linux with Python 3.

14. Limitations

- The application currently uses a local JSON file instead of a relational database.
- The interface is command-line based and does not provide a graphical user interface.
- It is intended for small-scale local record management rather than multi-user access.
- Advanced authentication, role-based access, and database-level security are not included.

15. Future Enhancements

- Add a relational database such as MySQL or PostgreSQL.
- Add stronger validation for Student ID, age, and email.
- Add sorting and filtering options.
- Add export/import features such as CSV.
- Develop a graphical or web-based user interface.
- Add user authentication and role-based access control.
- Add automated test cases for each record-management operation.

16. Conclusion

The Student Record Management System successfully demonstrates the integration of core Python programming concepts into a single working console application. It provides the required menu-driven operations for adding, viewing, searching, updating, and deleting student records, while using functions, exception handling, and JSON-based File I/O.

The project provides a practical foundation for understanding structured Python application development and can be extended in future with relational database connectivity, stronger validation, authentication, testing, and a graphical or web interface.

17. References

- Python 3 Documentation – Built-in types, functions, exceptions, and file handling.
- Python json module documentation – JSON encoding and decoding.
- Visual Studio Code documentation – Python development and terminal usage.

— End of Report —